SOFTOMATIC
TECHNOLOGY SERVICES

EXAMATE · SYSTEM DESIGN · ELITE + BATCH 1

Lesson 6 · Caching

> Opening the box labelled "cache" that Lessons 4 and 5 drew but never explained. Told through one story: the college canteen's ready shelf, and the cache in front of our short-link service sho.rt, from the first hit to the day Redis goes down.

MODULE
System Design · Part A
Foundations

TIME
About 85 minutes
plus 45 minutes of practice problems

YOU NEED
Lessons 4 and 5
hit rates, QPS, consistent hashing

YOUR ROUTE TODAY

01 A cache attacks its own database
02 Where caches live
03 Four strategies, eviction, TTL
04 Invalidation and stampedes
05 **Hot keys, Redis vs local**

LESSON 6 AT A GLANCE

What you will be able to do

Name every cache layer
Browser to database: what each is good and bad at.

Four strategies
Cache-aside, read-through, write-through, write-back: draw each, say what breaks.

Evict and expire
Trace LRU, LFU, FIFO by hand; TTLs; how Redis really does it.

Invalidate safely
Order of operations, and the race that still survives.

Stop stampedes
Locks, coalescing, stale-while-revalidate, jitter, early refresh.

Hot keys, local vs Redis
Detect and fix hot keys; choose a local cache, Redis, or both.

Subject / module	System Design / Part A: Foundations
Difficulty	Beginner to intermediate. You need Lesson 4's arithmetic (hit rates, QPS, the latency table) and Lesson 5's idea of consistent hashing.
Prerequisites	Lessons 1 to 5. Lesson 4 sized sho.rt at 7,716 redirects per second at peak, a 33 GB hot set, and showed that a 90% cache hit rate cuts database reads to about 772 per second. Lesson 5 routed redirects to 9 app servers behind a load balancer and put a CDN in front of static files. This lesson opens the box labelled "cache" that both lessons drew but never explained.
Next lesson	Lesson 7: Scaling databases: replication, sharding, consistent hashing
Companies covered	Amazon, Microsoft, Google, Uber, Flipkart, Atlassian, Adobe, Swiggy, Zomato, PhonePe, Razorpay, Walmart
Commands and configs	Real Redis commands (SET … EX, SET … NX EX, EXPIRE, TTL), the maxmemory and maxmemory-policy settings, nginx proxy_cache_lock, and HTTP Cache-Control: stale-while-revalidate.
Checklist	☐ where caches live ☐ cache-aside, read-through, write-through, write-back ☐ eviction (LRU, LFU, TTL) ☐ cache invalidation ☐ thundering herd and cache stampede ☐ hot keys ☐ Redis vs local cache
Citations	[S#] = a page opened in full (all listed on the Sources page). Practice problems and quiz questions are original and unlabelled.

SOURCE-CONFIRMED a dated first-person report naming the company and the question, Oct 2024 to Oct 2026, opened in full. **CANDIDATE-REPORTED** compilations, undated posts, older reports. **PATTERN-ONLY** commonly asked, no company source, so no company name.

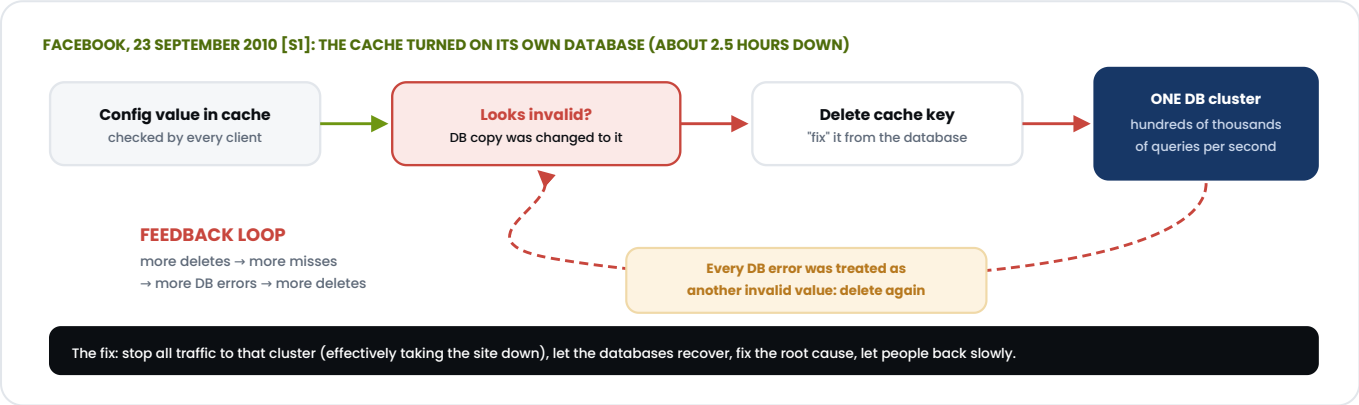
AN HONEST NOTE ON THE EVIDENCE

Caching reaches interviews in two shapes. The most common is **"implement an LRU cache"** as a coding or low-level-design question; for example, an Amazon software-engineer candidate in Hyderabad (reported June 2026) was pressed on combining a hash map with a doubly linked list and on how eviction works [S10]. Writing that code is LLD Lesson 19; this lesson explains the policy those questions rest on. The second shape is **conceptual follow-ups in a design round**: a Razorpay candidate (April 2025) was asked what Redis was used for, whether stale data is a concern for the cache layer, and what happens if Redis goes down [S9]. No dated caching question was found for Uber, Swiggy or Flipkart software-engineer loops in the window, so their names do not appear below.

23 SEPTEMBER 2010, FACEBOOK

The cache was supposed to protect the database. It attacked it.

Facebook was down or unreachable for about **2.5 hours**, which its engineering post called the worst outage in over four years [S1].



An automated system checked configuration values sitting in the cache; if a cached value looked invalid, it replaced it with the value from the database. Then someone changed the **database copy** to a value the system read as invalid. Every client saw it and tried to "fix" it, and one database cluster received **hundreds of thousands of queries per second** [S1]. Worse, every database error was treated as another invalid value, which deleted the cache key again, which caused more queries: a **feedback loop** [S1].

PAUSE AND PREDICT

How did engineers finally stop the loop?

Answer: They stopped traffic to that database cluster, which effectively took the site down, let the databases recover, fixed the root cause, and then slowly let people back on [S1].

Just for fun. The cache was supposed to protect the database. Instead it became the world's most efficient tool for attacking it.

INTERVIEW MODE: SAY IT LIKE THIS

- A cache makes reads fast and protects the database only while it is healthy and correct.
- A good design states what is cached, how long it stays, how it is updated or removed when data changes, and what protects the database when many requests miss at the same moment.

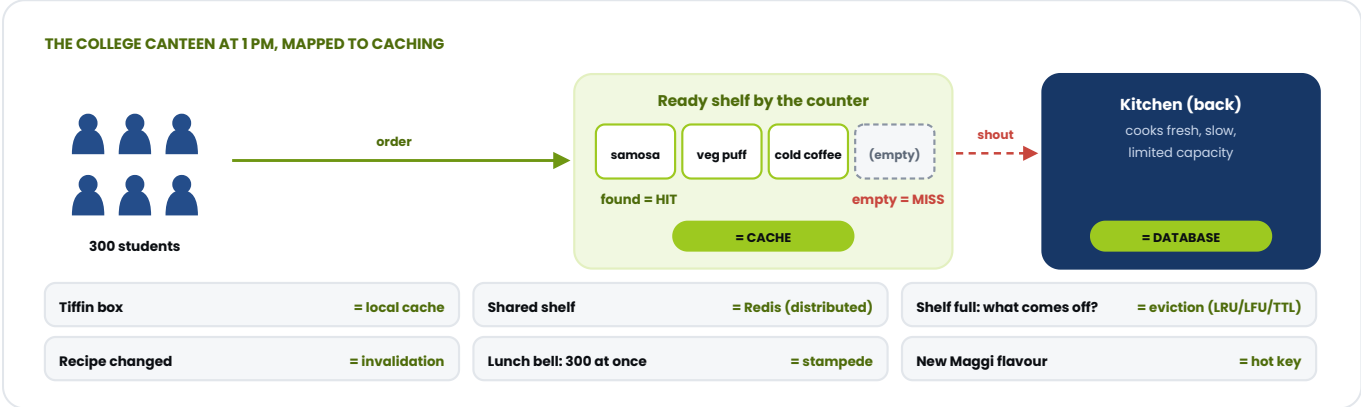
MEMORY HOOK: THE ONE SENTENCE

"A cache is a copy, and every copy needs a plan for when it is wrong and when it is empty." This lesson is that plan.

BEFORE THE JARGON

The college canteen's ready shelf

Picture the canteen at 1 pm. The kitchen at the back cooks everything fresh, but each dish takes minutes. So the canteen keeps a ready shelf next to the counter with the most-ordered dishes: samosas, veg puffs, cold coffee.



The kitchen is the database. It holds the truth. It is slow and can only cook so much at once. **The ready shelf is the cache.** It holds copies of popular items close to the counter. Finding the item on the shelf is a **cache hit**; finding an empty spot and shouting to the kitchen is a **cache miss**.

The shelf is small. When it is full and a new dish arrives, something must come off: the dish nobody has touched for the longest time (**LRU**), the one ordered least often (**LFU**), or anything older than 30 minutes (a **TTL**, time to live). That decision is **eviction**.

Where the shelf lives. Your own tiffin box is a **local cache**: fastest, but only you use it, and nobody updates it when the menu changes. The shared shelf is a **distributed cache** like Redis: one trip away, but everyone sees the same copy.

The recipe changes. The shelf still holds old samosas. Removing or replacing them is **cache invalidation**.

How the shelf gets filled. The counter person checks the shelf and, on a miss, fetches from the kitchen and puts one on the shelf (**cache-aside**). Or the shelf has its own runner who fetches automatically (**read-through**). When the kitchen cooks a batch it can stock the shelf at the same time (**write-through**), or the counter takes orders now and tells the kitchen later in one batch (**write-back**).

The lunch bell. At 1 pm the samosa tray empties and 300 students turn to the kitchen at once, all asking for the same samosa: a **cache stampede**, a form of **thundering herd**. The fix is one runner fetching one tray while everyone waits a few seconds. **The one dish everyone wants.** The day a new Maggi flavour launches, every order is for the same item and one spot on the shelf has a queue of 200: a **hot key**.

WHERE THE PICTURE STOPS WORKING

A samosa disappears when taken. Data does not.

WHAT GOES WRONG

Where the analogy breaks. Taking a samosa removes it; reading from a cache leaves the copy for the next reader, so one cached item can serve thousands of requests. A stale samosa is visibly cold, but **stale data looks exactly like fresh data**, which is why invalidation bugs are hard to spot. And a database has no idea a cache exists: unless you design it, nothing tells the cache that data changed.

TINY CHALLENGE

Which is worse, a shelf that is empty (all misses) or a shelf that confidently serves last week's samosas? Answer: Usually the stale shelf: empty is slow but correct, stale is fast and wrong. Hold that thought for Concept 5.

CANTEEN	SYSTEM DESIGN
Kitchen	Database (source of truth)
Ready shelf	Cache
Found on the shelf / shout to kitchen	Hit / miss
What comes off a full shelf	Eviction (LRU, LFU) and expiry (TTL)
Your tiffin box	Local (in-process) cache
The shared shelf	Distributed cache (Redis)
Recipe changed, old samosas	Stale data, invalidation
Counter fetches on a miss	Cache-aside
Shelf has its own runner	Read-through
Lunch bell, 300 at once	Stampede / thundering herd
New Maggi flavour	Hot key

MEMORY HOOK: SHELF, KITCHEN, BELL

Shelf = cache, **kitchen** = truth, **bell** = stampede. If you can tell the canteen story, you can answer half of this lesson.

INTERVIEW MODE: SAY IT LIKE THIS

- A cache holds copies of frequently read data in fast memory close to where it is needed.
- Because it is smaller than the data, it needs an eviction policy; because it holds copies, it needs invalidation; and because many requests can miss together, it needs protection for the database behind it.

EVERY WORD, ONE LINE (1 OF 2)

Vocabulary primer

Every term is defined before it is used. Terms from Lessons 2 to 5 (QPS, peak, p99, CDN, origin, DNS TTL, consistent hashing, load balancer, reverse proxy, idempotent) are reused without re-definition.

Cache
A small, fast store that keeps copies of data so later reads skip the slower original source.

Cache hit / miss
Hit: found in the cache. Miss: not found, so fetched from the source of truth.

Miss penalty
The extra time a miss costs compared with a hit.

Hot set (working set)
The part of the data that is read often; what a cache should hold (Lesson 4).

Cache warming
Loading likely-needed data before traffic arrives.

Distributed (shared) cache
A cache running as its own network service, shared by all app servers, such as Redis or Memcached.

Cache-aside (lazy loading)
The application checks the cache; on a miss it reads the database and writes the result into the cache itself.

Write-through
Every write updates the cache and the database as one operation; the write is done only when both are.

Write-around
Writes go straight to the database and skip the cache; the cache fills only on later reads.

Source of truth (backing store)
The authoritative data, usually a database. The cache is not the source of truth (except briefly in write-back).

Hit rate
Hits ÷ (hits + misses), as a percentage.

Key / value
The name used to look an item up (for example link:ab12cde) and the data stored under it.

Cold / warm cache
Cold: empty, most reads miss. Warm: already holds the hot set.

Local (in-process) cache
A cache inside the application's own memory, private to one process.

Redis
An open-source in-memory data store widely used as a distributed cache; commands look like GET key and SET key value.

Read-through
The application asks only the cache; on a miss the cache layer loads from the database by itself.

Write-back (write-behind)
Writes go to the cache first and reach the database later, asynchronously, often in batches.

EVERY WORD, ONE LINE (2 OF 2)

Eviction, invalidation, stampedes

Eviction
Removing entries because the cache is full.

LRU / LFU / FIFO
Evict the least recently used / least frequently used / first inserted entry.

maxmemory, maxmemory-policy
Redis's memory cap and the eviction policy applied when it is reached [S2].

Cache invalidation
Removing or updating cached copies when the underlying data changes.

Negative caching
Caching the fact that an item does not exist, so repeated lookups skip the database.

Cache stampede (dog-piling)
A popular entry expires or is evicted, many requests miss together, and all recompute it at once.

Lock (here)
A short-lived marker, often a Redis key set with SET … NX EX, that lets one worker rebuild an entry [S3].

Jitter
A small random amount added to a time value so that many things do not happen at the same instant.

Probabilistic early expiration (XFetch)
As expiry approaches, each request has a small, growing chance of refreshing the entry early [S6].

Eventual consistency (here)
Cache and database may disagree for a while but agree once updates stop and the cache catches up.

Expiration
Removing entries because their time limit passed, whether or not the cache is full.

TTL (time to live)
How long an entry may stay before it expires; in Redis set with EXPIRE or SET … EX and read with TTL [S3, S4].

Stale data
A cached copy that no longer matches the source of truth.

Race condition
A bug where the result depends on the timing of two overlapping operations.

Thundering herd
Many clients released at once by one event, all competing for the same resource.

Request coalescing (single-flight)
Only one request per key goes to the database; duplicates wait for its result.

Lease
Facebook's variant: memcached hands a token to the one client allowed to fill a missing key [S5].

Stale-while-revalidate
Serve an expired copy for a short grace period while one background request fetches a fresh one [S7].

Hot key
One key receiving a disproportionate share of traffic, overloading the cache node that holds it; Facebook observed a single key accounting for 20% of a server's requests [S5].

INTERVIEW MODE: SAY IT LIKE THIS

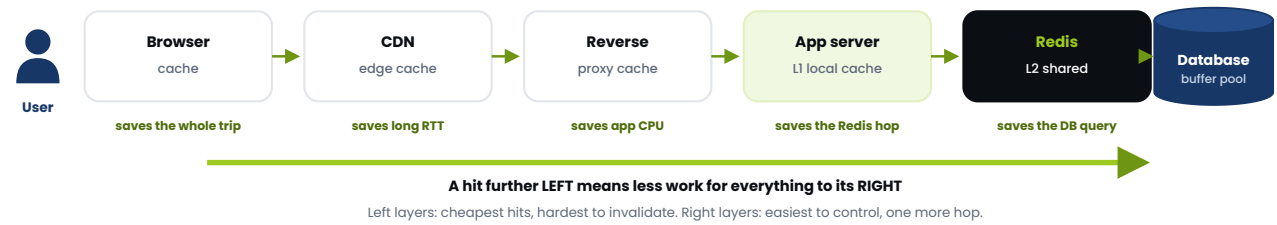
- I say cache-aside when the application manages the cache, read-through and write-through when the cache layer talks to the database itself, and write-back when the database is updated later.
- I say eviction for removal because the cache is full and expiration for removal because a TTL ran out, and I always name the source of truth.

CONCEPT 1 · WHERE CACHES LIVE

"Add a cache" is not an answer until you say where

Eight steps, bottom-up. The running example is sho.rt from Lessons 1 to 5: **7,716 redirects per second at peak**, a **33 GB hot set**, **9 app servers** behind a load balancer. Each cache layer removes a different cost.

SOURCE DIAGRAM 1, REDRAWN: EVERY PLACE A CACHE CAN LIVE



LAYER	EXAMPLE	WHAT IT SAVES	TYPICAL COST	MAIN WEAKNESS
Browser / app	Cache-Control: max-age=..., in-app memory	The whole network trip	0 network	You can't invalidate it from the server; users keep stale copies until expiry
CDN / edge	CDN nodes near users (Lesson 5)	Long-distance RTT and origin load	One short RTT	Mostly for public, cacheable responses; purges take time
Reverse proxy	nginx proxy_cache	App server CPU	One hop	Shared by everyone behind it; per-user pages are hard
Application, local	In-process hash map or library	Network hop to Redis	Nanoseconds to microseconds (an in-memory lookup)	Each of 9 servers has its own copy: 9 copies to invalidate, 9 cold starts
Application, distributed	Redis, Memcached	Database query	One in-datacentre RTT, often well under a millisecond (assumption)	Network dependency; one more system to run and fail
Database internal	Buffer pool, query cache	Disk reads	Inside the DB	You don't control it; it doesn't reduce connection or CPU load from the app

CONCEPT 1 · THE HIT-RATE SUM

Hit rate is your cache's report card

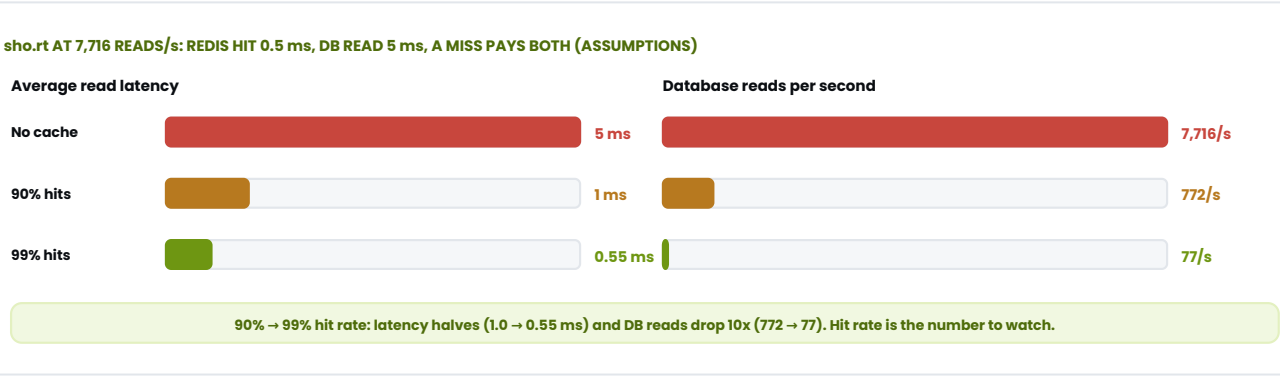
Worked example: average read latency for sho.rt. Assumptions (labelled): a Redis hit costs **0.5 ms**; a database read costs **5 ms**; on a miss you pay both (check Redis, then the database).

PAUSE AND PREDICT

Before reading on: what is the average read time at a 90% hit rate?

Answer: 1.0 ms: 0.9 × 0.5 + 0.1 × (0.5 + 5) = 0.45 + 0.55.

HIT RATE	CALCULATION	AVERAGE READ	DATABASE READS PER SECOND
No cache	every read 5 ms	5 ms	7,716
90%	0.9 × 0.5 + 0.1 × (0.5 + 5) = 0.45 + 0.55	**1.0 ms**	10% of 7,716 = **772** (Lesson 4)
99%	0.99 × 0.5 + 0.01 × 5.5 = 0.495 + 0.055	**0.55 ms**	about **77**



Going from 90% to 99% **halves latency and cuts database reads by 10×**. Hit rate is the number to watch.

Just for fun. Hit rate is your cache's CGPA. Nobody cares how fancy the cluster is if it's sitting at 40%.

TINY CHALLENGE

A miss with no cache layer at all costs 5 ms. With a 0% hit rate behind Redis, what does each read cost? Answer: 5.5 ms: you pay the Redis check and then the database. A cache that never hits makes things slower.

MEMORY HOOK: LEFT SAVES MORE

The further **left** (closer to the user) a hit happens, the less work everything to its right does. The further **right**, the easier it is to control.

INTERVIEW MODE: SAY IT LIKE THIS

- Caches can live in the browser, the CDN, a reverse proxy, the application process, a distributed cache like Redis, or inside the database. Each removes a different cost.
- With a 0.5 ms cache and a 5 ms database, a 90% hit rate gives about 1 ms average reads and cuts database load by 90%; raising the hit rate to 99% cuts database reads another tenfold.

CONCEPT 2 · CACHE-ASIDE (LAZY LOADING)

The default: the application owns the cache

How it works. On a read, check the cache; on a miss, read the database and fill the cache. On a write, update the database first, then delete the key.

```
read(key):
    value = cache.get(key)
    if value is not None:           # hit
        return value
    value = db.query(key)           # miss
    cache.set(key, value, ttl)      # fill for next time
    return value

write(key, new_value):
    db.update(key, new_value)       # 1. source of truth first
    cache.delete(key)               # 2. then invalidate
                                    #    (next read refills)
```

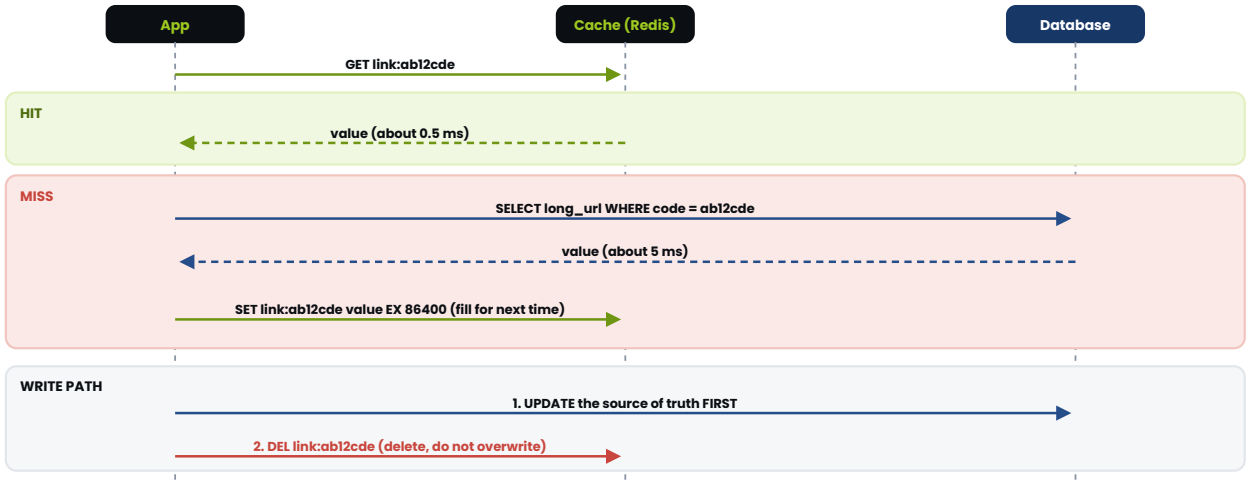
Why it is the default. The cache holds only what is actually read (efficient memory use), and if the cache is down the app can still read from the database (slower, but correct). The application controls everything, so it works with any database.

PAUSE AND PREDICT

The cache has just restarted and is empty. What happens to the first read of every key?

Answer: Every one misses and goes to the database: a cold start. Concept 6 explains why that can hurt.

SOURCE DIAGRAM 2, REDRAWN: CACHE-ASIDE READ AND WRITE PATHS



Source diagram 2, redrawn. The application owns both paths; the database is always written first and the cache key is deleted, not overwritten.

CONCEPT 2 · REAL COMMANDS AND WHAT GOES WRONG

Four Redis commands and one classic leak

Redis commands for sho.rt:

```
SET link:ab12cde "https://example.com/very/long/url" EX 86400   # cache for 24 hours
GET link:ab12cde
TTL link:ab12cde                                                  # seconds left
DEL link:ab12cde                                                  # invalidate on edit
```

Note: a plain SET on an existing key discards its old TTL [S3], so re-caching a key without EX makes it live forever. That is a classic memory-leak bug.

WHAT GOES WRONG

- **First read is always a miss** (cold start); after a cache restart, every key misses at once (see Concept 6).
- **Stale data window:** between the database update and the cache delete, readers see the old value; and a subtle race can put stale data back (Concept 5).
- **Every service must follow the rules.** One service that writes to the database without deleting the cache key leaves stale data until the TTL expires.

TINY CHALLENGE

You run SET link:ab12cde "new-url" (no EX) on a key that had 3 hours left. What does TTL link:ab12cde return now? Answer: -1: the key has no expiry any more [S3]. It will live until evicted or deleted.

MEMORY HOOK: ASIDE = APP DOES IT

Cache-**aside**: the cache sits **aside**, and the app walks over to it. **DB first, then DEL.**

INTERVIEW MODE: SAY IT LIKE THIS

- In cache-aside the application reads the cache, falls back to the database on a miss and fills the cache; on writes it updates the database and then deletes the cache key.
- It caches only what is read and degrades to the database if the cache fails, at the cost of a miss on first access and a short stale window.

CONCEPT 3 · READ-THROUGH, WRITE-THROUGH, WRITE-BACK, WRITE-AROUND

Who talks to the database, and when?

Cache-aside puts the application in charge. The other strategies move that work into the cache layer, or change when the database hears about a write.

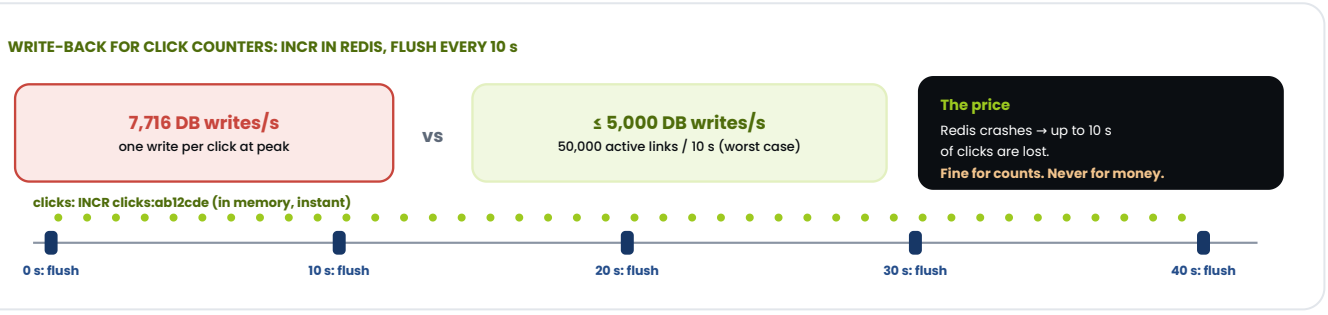


STRATEGY	READ PATH	WRITE PATH	STRENGTH	WEAKNESS
Read-through	App asks cache; cache loads from DB on miss	(paired with one of the write strategies)	Simpler app code; loading logic in one place	Cache layer must know how to read the DB; first read still misses
Write-through	Reads hit a cache that is always populated for written keys	Write to cache and DB synchronously; done only when both succeed	Cache is fresh for written data	Every write pays both latencies; writes data that may never be read
Write-back (write-behind)	Reads hit the cache	Write to cache only; flush to DB later in batches	Very fast writes; batches absorb write spikes	Data loss if the cache node dies before flushing; DB is temporarily behind
Write-around	Cache-aside reads	Write to DB only; cache filled by later reads	Avoids filling cache with write-once data	First read after a write misses

CONCEPT 3 · WRITE-BACK, PRICED HONESTLY

Fast writes, with a fuse

Worked example: write-back for view counters. sho.rt counts clicks per link. At peak, **7,716 clicks/s**; writing each to the database means 7,716 writes/s. With write-back, increment a counter in Redis (`INCR clicks:ab12cde`) and flush every 10 seconds: one database write per active link per 10 s. If 50,000 links are active, that is 50,000 / 10 = **5,000 writes/s worst case**, but most clicks hit a few viral links, so in practice far fewer. The price: if Redis crashes, you lose up to 10 seconds of clicks. Acceptable for click counts; **never acceptable for payments or bank balances**.



Just for fun. Write-back for payments is like the canteen uncle saying "I'll write your UPI payment in the notebook later." Bro, the notebook is on fire.

PAUSE AND PREDICT

Which strategy would you choose for user profiles that are read 1,000 times for every update?

Answer: Cache-aside (or read-through) with delete-on-write: reads dominate, writes are rare, and a stale profile for a few seconds is acceptable. Write-through would also work but adds latency to every profile update for little gain.

TINY CHALLENGE

A write-back cache flushes every 30 s and its node dies 25 s after the last flush. How much is lost?
Answer: Up to 25 seconds of writes: everything since the last flush.

MEMORY HOOK: THROUGH, BACK, AROUND

Write-**through**: through the cache **to** the DB, now. Write-**back**: the DB hears **back** later. Write-**around**: walk **around** the cache.

INTERVIEW MODE: SAY IT LIKE THIS

- Read-through moves the loading logic into the cache layer. Write-through updates cache and database together, keeping written data fresh but slowing writes.
- Write-back writes to the cache and flushes later, giving fast writes at the risk of losing unflushed data, so I use it only for loss-tolerant data like counters. Write-around writes straight to the database and lets reads fill the cache.

CONCEPT 4 · EVICTION (LRU, LFU, FIFO) AND EXPIRY (TTL)

The shelf is full. What comes off?

Eviction runs when the cache is full; expiry runs when time is up. They are different mechanisms, and Redis uses both.

Hand trace. Cache capacity **3 entries**. Access sequence: **A B C A D B E A**. LFU breaks ties by evicting the oldest entry.

PAUSE AND PREDICT

Before you look: which policy gets the most hits on this sequence?

Answer: FIFO and LFU tie with **2** hits each; LRU gets only **1**.

CAPACITY 3, ACCESS SEQUENCE A B C A D B E A (✓ = hit)

Step	LRU (oldest → newest use)				FIFO (oldest → newest insert)				LFU (with use counts)			
1 A	A			miss	A			miss	A1			miss
2 B	A	B		miss	A	B		miss	A1	B1		miss
3 C	A	B	C	miss	A	B	C	miss	A1	B1	C1	miss
4 A	B	C	A	✓	A	B	C	✓	A2	B1	C1	✓
5 D	C	A	D	miss	B	C	D	miss	A2	C1	D1	miss
6 B	A	D	B	miss	B	C	D	✓	A2	D1	B1	miss
7 E	D	B	E	miss	C	D	E	miss	A2	B1	E1	miss
8 A	B	E	A	miss	D	E	A	miss	A3	B1	E1	✓
	1 hit				2 hits				2 hits			

FIFO beat LRU here. No policy wins on every access pattern.

Every row recomputed by a simulator when this PDF was built. Green cells are the hit.

Surprised that FIFO beat LRU here? No policy is best for every access pattern. **LRU** wins when recent items are likely to be reused soon (most web traffic); **LFU** wins when some items are popular for a long time; both can be fooled. LFU also needs **ageing**, or yesterday's viral item stays forever.

POLICY	EVICTS	WINS WHEN	FOOLED BY
LRU	Least recently used	Recent items are likely to be reused soon (most web traffic)	A one-off scan of many keys pushes out the regulars
LFU	Least frequently used	Some items stay popular for a long time	Yesterday's viral item, unless counts age
FIFO	First inserted	Simplest to build; fine when every item is equally likely	A popular item that was simply inserted early

CONCEPT 4 · HOW REDIS REALLY DOES IT

Redis does not keep an exact LRU list

Redis samples a few keys at random and evicts the one idle longest (`maxmemory-samples`, default 5) [S2]. LFU is also approximate, using a small probabilistic counter per key that decays over time (by default the counter saturates around a million accesses and decays every minute) [S2].



POLICY [S2]	EVICTS	USE WHEN
noeviction	Nothing; writes fail when full	Redis used as a store you cannot lose data from
allkeys-lru	Least recently used, any key	General cache; Redis's docs call it a good default when a subset is far more popular [S2]
allkeys-lfu	Least frequently used, any key	Long-lived popularity
volatile-lru / volatile-lfu	Same, but only keys with a TTL	Mixed: some keys must never be evicted
volatile-ttl	Key with the shortest remaining TTL	You encode priority in TTLs
allkeys-random / volatile-random	Random	Uniform access

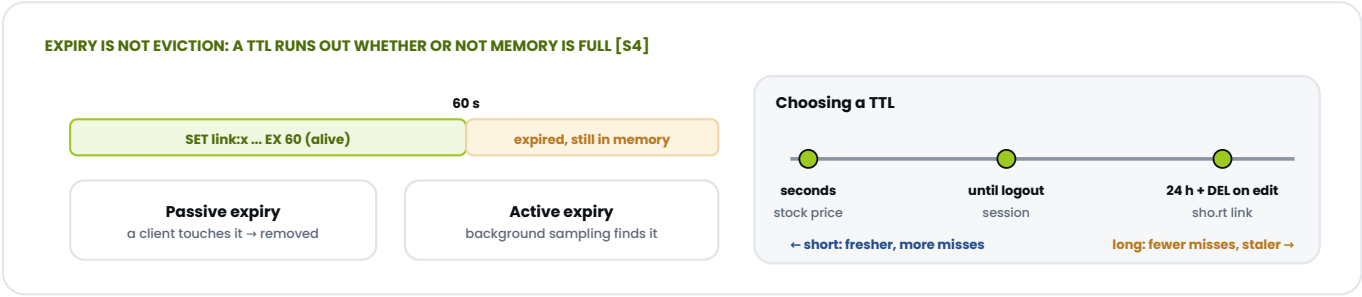
WHAT GOES WRONG

volatile-* policies only touch keys that have a TTL. If half your keys were written without one, those can never be evicted, and once only they remain, writes fail as if the policy were noeviction (Practice problem 6).

CONCEPT 4 · TTLS

Expired is not the same as gone

TTL expiry in Redis. Keys expire two ways: **passively**, when a client accesses an expired key, and **actively**, through periodic background sampling [S4]. So an expired key may still use memory for a short time until it is noticed.



Choosing a TTL. Short TTL: fresher data, more misses. Long TTL: fewer misses, staler data. For sho.rt, a link rarely changes, so **24 hours plus explicit invalidation on edit**. A stock price: seconds. A user's session: until logout.

DATA (FROM THIS LESSON)	TTL	WHY
sho.rt link	24 h ± 1 h jitter, plus DEL on edit	Rarely changes; edits handled by invalidation
Stock price	Seconds	Changes constantly; stale is wrong fast
User session	Until logout	Must live exactly as long as the login
"Not found" (negative cache)	60 s	A newly created item must appear quickly
Hot key in the L1 local cache	1–5 s	Bounds staleness across 9 private copies

TINY CHALLENGE

Trace LRU with capacity 2 on X Y X Z. Which key is evicted when Z arrives? Answer: Y: X was used more recently (step 3).

MEMORY HOOK: RECENCY VS FREQUENCY

LRU asks "when did you last come?" **LFU** asks "how often do you come?" **TTL** asks "how long have you been here?"

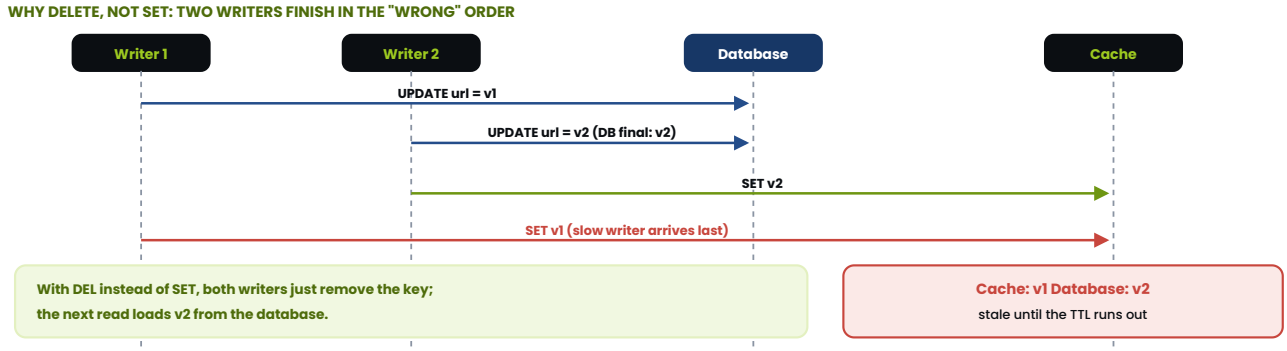
INTERVIEW MODE: SAY IT LIKE THIS

- Eviction removes entries when the cache is full; expiry removes them when their TTL passes. LRU evicts the least recently used, LFU the least frequently used with ageing, FIFO the oldest inserted; none is best for every pattern.
- Redis approximates LRU and LFU by sampling a few keys, offers policies such as allkeys-lru and volatile-ttl, and expires keys both lazily on access and actively in the background.

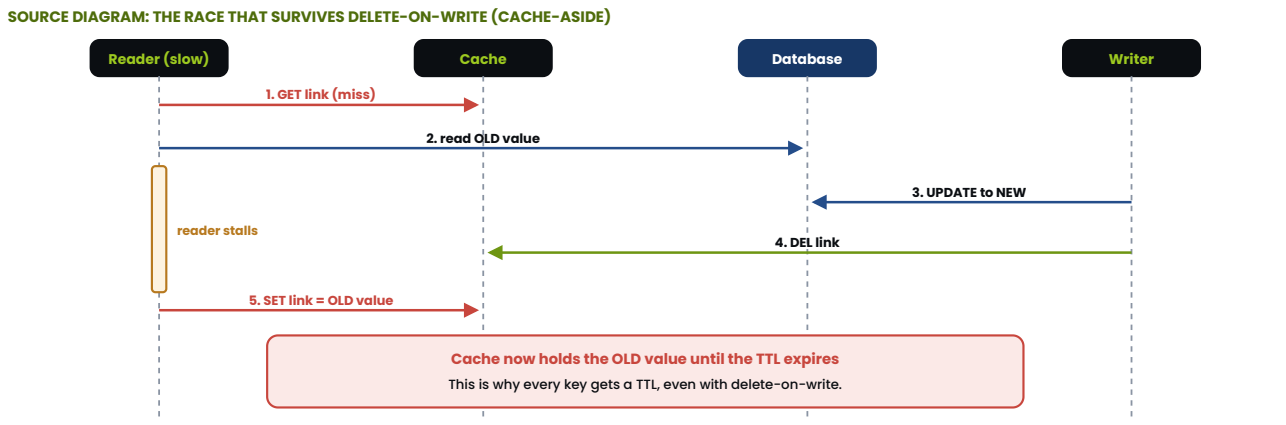
CONCEPT 5 · CACHE INVALIDATION

Update the database, then delete the key

The rule: **update the database first, then delete the cache key (don't overwrite it)**. Deleting is safer than setting because two concurrent writers that each set the cache can leave it with the older value.



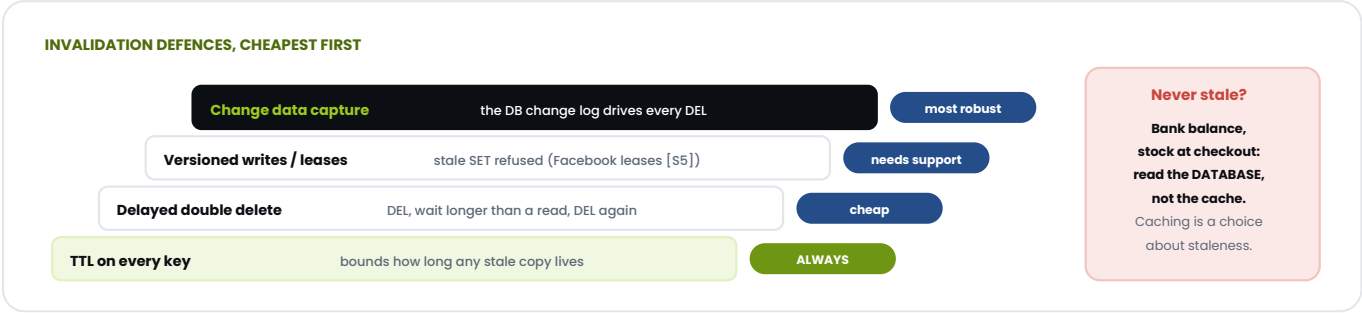
The race that still survives (cache-aside). Even with delete-on-write, a slow reader can put the old value back:



Source diagram, redrawn. A slow reader that fetched the old value before the update writes it into the cache after the writer's delete, so stale data survives until the TTL runs out.

CONCEPT 5 · DEFENCES, IN INCREASING COST

Bound the damage, then shrink it



A TTL on every key. Bounds how long any stale copy can live. Always do this.

Delayed double delete. Delete, then delete again after a short delay (longer than a typical read), to remove a value a slow reader put back.

Versioned writes / leases. Facebook's memcache leases give a token to the client that missed; if the key was invalidated in the meantime, the token is rejected and the stale set is refused [S5].

Change data capture. Invalidate from the database's own change log, so every committed change triggers a delete, no matter which service wrote it.

WHEN STALE IS NOT ALLOWED AT ALL

A bank balance, inventory at checkout: **don't serve that read from the cache**; read the database, or design the data path so the cache is updated in the same transaction scope. Caching is a choice about acceptable staleness.

Just for fun. There are only two hard things in computer science: cache invalidation, naming things, and off-by-one errors.

TINY CHALLENGE

Your delete-on-write is in place, yet a user still sees an old URL for a few minutes once a week. Name the likely cause and the cheapest fix. Answer: The slow-reader race. A TTL bounds it; a delayed second delete usually removes it.

MEMORY HOOK: DB, DEL, TTL

Say it in order: **DB** first, then **DEL**, and a **TTL** on everything as the safety net.

INTERVIEW MODE: SAY IT LIKE THIS

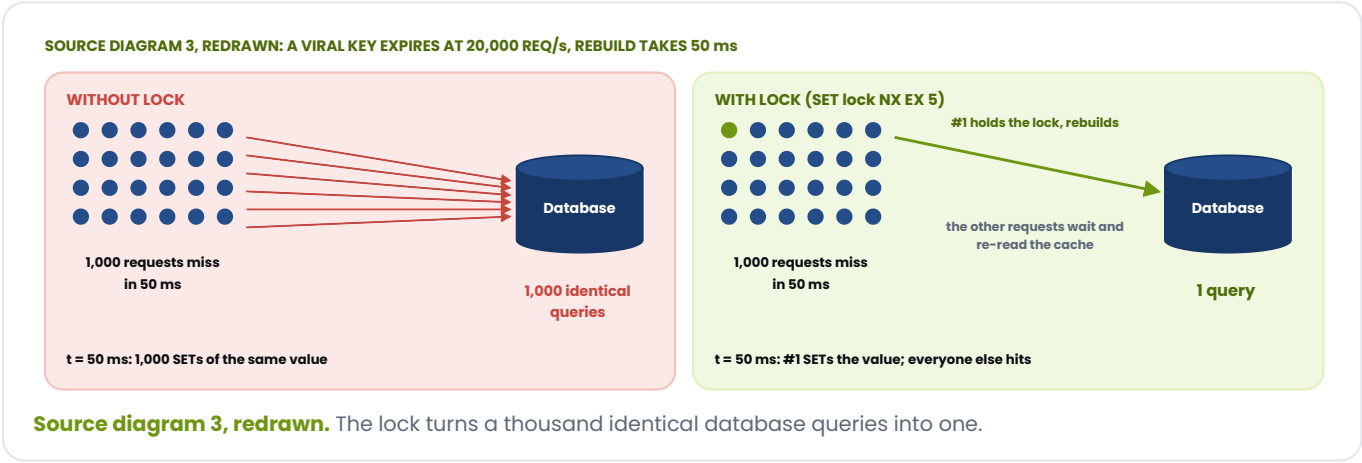
- On writes I update the database first and then delete the cache key rather than overwriting it.
- A race can still let a slow reader re-insert the old value, so every key has a TTL as a bound; when stricter freshness is needed I use delayed double deletes, versioned writes or leases, or invalidation driven by the database change log. Data that must never be stale is read from the source of truth.

CONCEPT 6 · THUNDERING HERD AND CACHE STAMPEDE

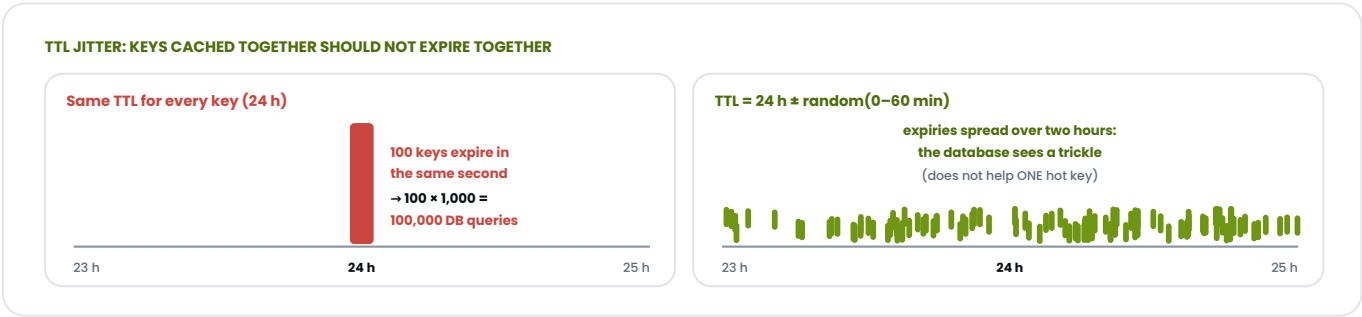
The lunch bell rings and everyone misses at once

A popular key expires (or the cache restarts). Thousands of requests miss in the same instant, all query the database, and all write the same value back.

Worked numbers. A viral sho.rt link gets **20,000 requests/s**. Rebuilding the entry takes **50 ms** (database read under load). During those 50 ms, about 20,000 × 0.05 = **1,000 requests** miss and hit the database for the same row. If 100 popular keys expire at the same second because they were all cached at the same time with the same TTL, that is **100,000 database queries in a burst**.



Source diagram 3, redrawn. The lock turns a thousand identical database queries into one.



CONCEPT 6 · SEVEN FIXES (COMBINE THEM)

One rebuild, not a thousand

FIX	HOW	TRADE-OFF
TTL jitter	TTL = 24 h ± random(0–60 min), so keys cached together don't expire together	Doesn't help a single hot key
Lock / single-flight	First miss acquires `SET lock:link:ab12cde 1 NX EX 5` [S3]; it rebuilds; others wait briefly and re-read the cache	Waiters add latency; lock must expire if the holder dies (the EX)
Request coalescing in-process	Within one server, concurrent misses for the same key share one database call	Still one call per server (9 servers → 9 calls)
Stale-while-revalidate	Keep serving the expired value for a grace period while one background request refreshes it; HTTP standardises this as Cache-Control: stale-while-revalidate [S7]	Users see slightly stale data for the grace period
Probabilistic early expiration (XFetch)	Each read near expiry refreshes early with a probability that rises as expiry approaches, so usually one request refreshes before expiry [S6]	Small extra recomputation
Leases	memcached issues a fill token to one client per key at most every 10 s by default; others are told to wait [S5]	Requires cache support
Proxy-level locks	nginx proxy_cache_lock on lets one request populate a new cache element while others wait (default lock timeout 5 s) [S8]	Proxy-layer only

SITUATION	REACH FOR
Many keys cached at the same moment (nightly job, warm-up)	TTL jitter
One viral key expiring	Lock / single-flight, stale-while-revalidate, early refresh
Duplicate misses inside one app server	In-process request coalescing
HTTP responses at a proxy or CDN	stale-while-revalidate header, nginx proxy_cache_lock
Whole cache empty (restart)	Replicas, warming, rate limits and a circuit breaker on the DB

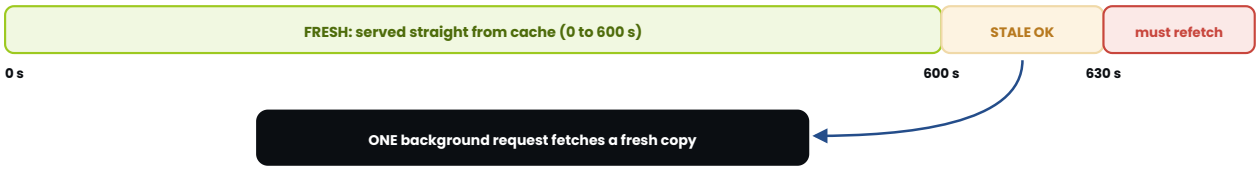
Facebook reports that with leases its peak database query rate in one experiment was **1.3K/s** [S5]; the point is not the number but that the herd was reduced to a trickle.

Cold restart. If Redis restarts empty, every key misses at once: a herd on the whole dataset. Defences: replicas (so one node's restart doesn't empty the cache), warming the hottest keys before taking traffic, and rate-limiting database access with a circuit breaker (Lesson 11) so the database survives the burst.

CONCEPT 6 · SAY IT IN THE INTERVIEW

Jitter, lock, stale, early

```
Cache-Control: max-age=600, stale-while-revalidate=30 [S7]
```



Users in the grace period get the slightly stale copy instantly; nobody waits, and the origin sees one refetch, not a herd.

PAUSE AND PREDICT

A lock holder crashes halfway through the rebuild. Do the waiters wait forever?

Answer: No: the lock was set with EX 5, so it expires after 5 seconds and another request takes over [S3].

TINY CHALLENGE

A train-enquiry API gets 6,000 requests/s for one route; rebuild takes 100 ms. Duplicate DB queries at expiry without a lock? Answer: 6,000 × 0.1 = 600. With a lock: 1.

MEMORY HOOK: J-L-S-E

Jitter the TTLs, **L**ock the rebuild, serve **S**tale while refreshing, refresh **E**arly. "Just Leave Samosas Early."

WHAT GOES WRONG

"Just increase the TTL." It only delays the stampede to a later moment; when the key finally expires, the same herd arrives.

INTERVIEW MODE: SAY IT LIKE THIS

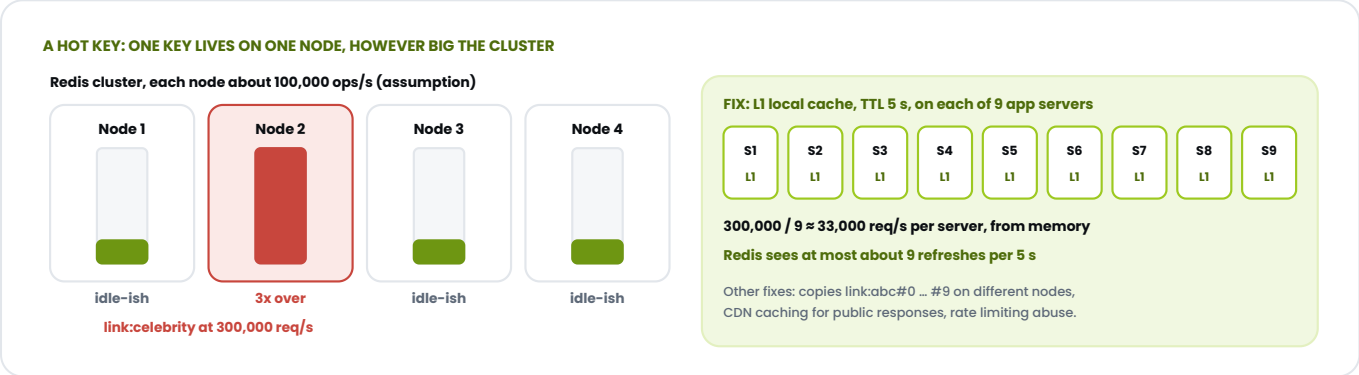
- A stampede happens when a popular key expires and many requests rebuild it at once.
- I add jitter to TTLs, let one request rebuild behind a short-lived lock or request coalescing, serve slightly stale data while refreshing in the background, or refresh probabilistically before expiry.
- For a cold cache I warm hot keys and protect the database with rate limits.

CONCEPT 7 · HOT KEYS

One key, one node, one very bad day

A distributed cache spreads keys across nodes (Redis Cluster uses hash slots; Lesson 5's consistent hashing is the same idea). A single key always lives on one node. If one key gets a huge share of traffic, that node saturates while the others sit idle. Facebook saw a single key account for **20% of a server's requests** [S5].

Worked numbers. Assume one Redis node handles about **100,000** simple operations per second (an order-of-magnitude assumption; measure your own). A celebrity's link goes viral at **300,000 requests/s**: 3× what its node can serve, even though the cluster as a whole has plenty of capacity.



Just for fun. A hot key is the one friend whose birthday post gets 10,000 comments while the rest of the group chat is dead silent. Same server, very different energy.

Detect. Per-key request counters in the app, sampling of commands, Redis's hot-key tooling where available, and alarms on one node's CPU being far above its peers.

PAUSE AND PREDICT

Will adding 10 more Redis nodes fix it?

Answer: No. The key still hashes to one node. You need to spread **that key**, not the cluster.

CONCEPT 7 · FOUR FIXES

Spread the key, not the cluster

1. **Local (L1) cache** in each app server with a very short TTL (say 1–5 s) for the hot key. With 9 app servers, each serves its share from memory: 300,000 / 9 ≈ **33,000 requests/s per server**, and Redis sees at most about one refresh per server per TTL: **9 requests every few seconds**. Staleness is bounded by the short TTL.
2. **Key replication.** Store copies as `link:abc#0 … link:abc#9` on different nodes and read a random suffix; writes must update all copies.
3. **CDN or proxy caching** for public responses (a redirect is perfect for this).
4. **Rate limiting** for abusive traffic on one key.

TINY CHALLENGE

A key is copied as #0 to #9 across nodes. What is the cost on every write to it? Answer: Ten writes (or deletes) instead of one, and a short window where copies disagree.

MEMORY HOOK: POCKET COPY

For the one dish everyone wants, every counter keeps **a few in its own pocket** for 5 seconds. That is the L1 hot-key fix.

	HOT KEY	BIG KEY
What it is	One key gets a huge share of requests	One value is huge (a giant list or hash)
Symptom	One node's CPU far above its peers	One node slow; large transfers on every read
Fix	L1 local cache, key replication, CDN, rate limit	Split it into smaller keys

WHAT GOES WRONG

"Consistent hashing will balance it." Consistent hashing balances **keys**, not **traffic per key** (Lesson 5). One key at 300,000 requests/s is still one key.

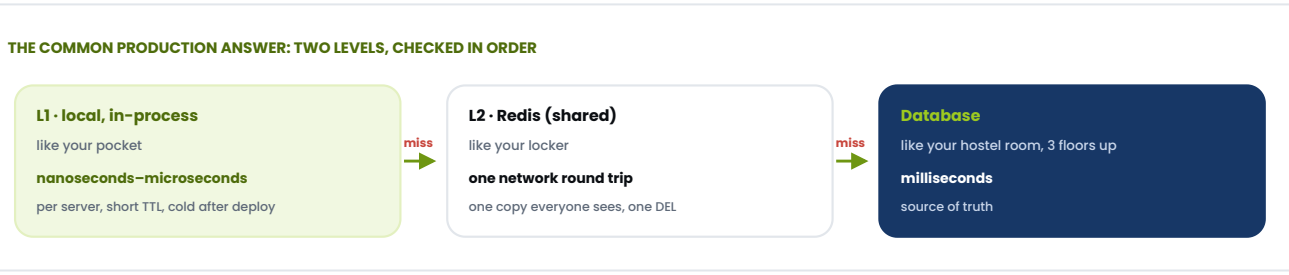
INTERVIEW MODE: SAY IT LIKE THIS

- A hot key overloads the single cache node that owns it even when the cluster has spare capacity.
- I detect it with per-key metrics and fix it with a short-TTL local cache on each app server, replicated copies of the key on several nodes, CDN caching for public responses, and rate limiting.

CONCEPT 8 · REDIS VS LOCAL CACHE (AND USING BOTH)

Pocket, locker, hostel room

	LOCAL IN-PROCESS CACHE	REDIS (DISTRIBUTED)
Latency	Fastest: memory lookup, no network	One network round trip
Shared across servers	No: each server has its own copy	Yes: one copy everyone sees
Invalidation	Hard: must reach every server (broadcast, or short TTL)	One DEL
Survives app restart/deploy	No: cold after every deploy	Yes (Redis keeps running)
Capacity	Limited by each server's heap; duplicated 9×	Scales by adding nodes; one copy
Failure mode	None external	Redis down → all servers miss → database load spike
Extra features	Simple key → value	TTLs, atomic INCR, locks with SET NX, sorted sets, pub/sub



Just for fun. L1 is your pocket, L2 is your locker, the database is your hostel room three floors up. You check them in that order, obviously.

The common production answer: two levels. L1 local cache with a short TTL for the hottest keys; L2 Redis for everything else; database behind both. Reads try **L1 → L2 → database**.

PAUSE AND PREDICT

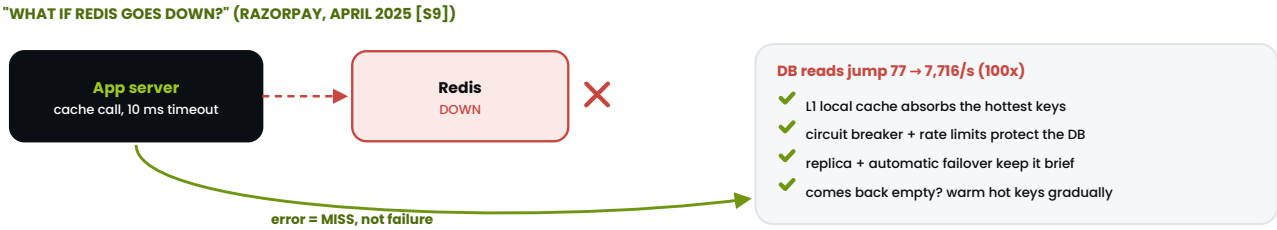
You deploy all 9 app servers at once. Which layer goes cold, and which survives?

Answer: Every L1 local cache is empty again; Redis (L2) keeps running, so the database is shielded (Scenario 1).

CONCEPT 8 · "WHAT IF REDIS GOES DOWN?"

Misses, not errors

Asked at Razorpay [S9]. The answer has four parts.



1. The app must treat **cache errors as misses**, not as failures, and use **short timeouts** on cache calls so a slow Redis doesn't slow every request.
2. All reads now hit the database; protect it with a **circuit breaker, rate limits**, and the **L1 cache** absorbing the hottest keys.
3. Run Redis with **replicas and automatic failover** so a node failure is brief.
4. When Redis comes back empty, **warm hot keys gradually** instead of letting a stampede refill it.

TINY CHALLENGE

Normally 99% of 7,716 reads/s are cached. Redis dies. How much does database read load multiply? Answer: From about 77 to 7,716 reads/s: about 100×.

MEMORY HOOK: CACHE = OPTIMISATION

The cache is an **optimisation**, not the source of truth. If removing it breaks correctness (not just speed), the design is wrong.

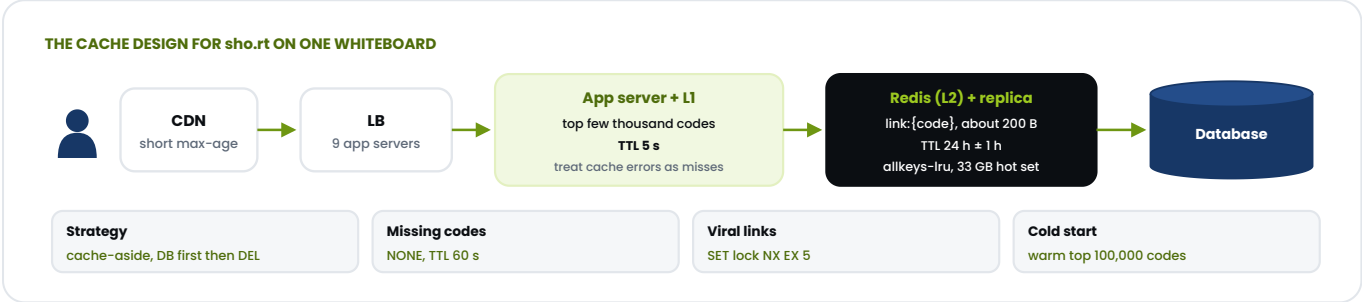
INTERVIEW MODE: SAY IT LIKE THIS

- A local cache is fastest but duplicated per server, hard to invalidate and cold after every deploy; Redis is shared, easy to invalidate and survives app restarts but adds a network hop and a dependency.
- I often use both: a short-TTL local cache for the hottest keys and Redis for the rest. If Redis fails, cache errors become misses with short timeouts, and the database is protected by circuit breakers, rate limits and the local cache while replicas fail over.

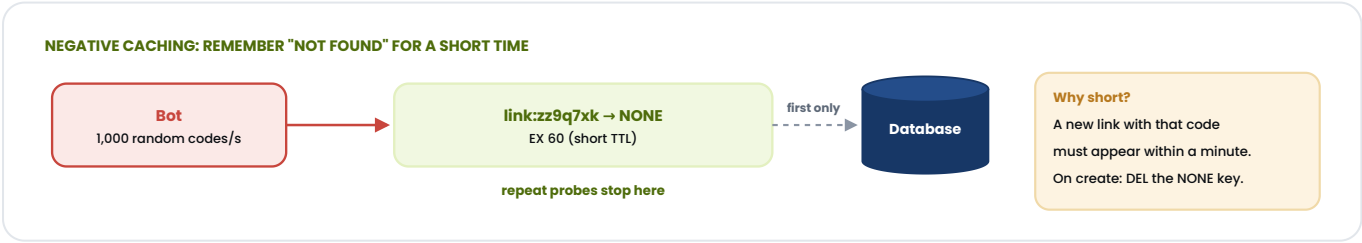
ONE WHITEBOARD, EVERY DECISION

The cache design for sho.rt

Requirements recap (from Lessons 1–5). 7,716 redirects/s at peak; 100 reads per write; 33 GB hot set; 9 app servers; links rarely change but can be edited or deleted; bots scan random codes that don't exist.



1. **What to cache.** Key `link:{code}` → long URL (about 100 bytes of value plus key and overhead; assume about 200 bytes per entry). Hot set 33 GB fits on one large node or a small cluster with replicas.
2. **Strategy.** Cache-aside with delete-on-write. Writes are about 77/s (1% of traffic), so write-through would add latency for little gain.
3. **TTLs and eviction.** TTL 24 h with ±1 h jitter. maxmemory set below the node's RAM, maxmemory-policy allkeys-lru (popularity is skewed) [S2].
4. **Negative caching.** A bot requests 1,000 random codes per second; each is a miss that hits the database. Cache "not found" as `link:{code} → NONE` with a short TTL (60 s), so repeated probes are cheap, but a newly created link with that code becomes visible within a minute. (On creating a link, also delete any NONE entry for its code.)
5. **Stampede protection.** Jittered TTLs for the bulk; for viral links, a lock with `SET lock:link:{code} 1 NX EX 5` [S3] so one request rebuilds while others retry the cache after a few milliseconds.
6. **Hot keys.** L1 in-process cache, TTL 5 s, for the top few thousand codes by request rate. A redirect response can also be cached at the CDN with a short max-age (Lesson 5).



STEP 7 AND 8: THE NUMBERS AND THE FAILURE PLAN

Every number on one table

QUANTITY	CALCULATION	RESULT
Database reads at 90% hit	7,716 × 0.10	about 772/s (Lesson 4)
Database reads at 99% hit	7,716 × 0.01	about 77/s
Average read latency at 99% (0.5 ms cache, 5 ms DB)	0.99 × 0.5 + 0.01 × 5.5	about 0.55 ms
Stampede on one viral link without a lock (20,000 req/s, 50 ms rebuild)	20,000 × 0.05	about 1,000 duplicate DB queries
Same with a lock	one rebuild	1 DB query
Hot link at 300,000 req/s with L1 on 9 servers	300,000 / 9	about 33,000 req/s per server from local memory; Redis sees about 9 refreshes per 5 s

STEP 8: THE FAILURE PLAN

Redis has a **replica with automatic failover**; the app treats cache timeouts (say **10 ms**) as misses; the database is protected by a **circuit breaker**; on cold start, **warm the top 100,000 codes** from yesterday's access log before taking full traffic.

INTERVIEW MODE: SAY IT LIKE THIS

- For the short-link service I cache code-to-URL mappings in Redis with cache-aside, delete on edit, a 24-hour jittered TTL and allkeys-lru.
- Unknown codes are negatively cached for a minute; viral links are protected by a rebuild lock and a five-second local cache on each app server.
- At a 99% hit rate the database sees about 77 reads per second, and the cache tier has replicas, short timeouts and a warming plan.

TINY CHALLENGE

A bot probed code ab99xyz, so link:ab99xyz → NONE (EX 60) was cached. 20 s later a user creates that link. If the create step forgets to delete the NONE key, how long does the new link look missing? Answer: Up to about 40 more seconds, until the NONE entry expires. With the DEL on create, it works at once.

THINGS THAT SOUND RIGHT AND ARE NOT

Common misconceptions

IT SOUNDS RIGHT	WHAT IS ACTUALLY TRUE
"Caching always makes things faster."	A low hit rate adds a network hop to every request and makes misses slower. Measure the hit rate.
"Update the cache on write."	Setting the cache on write can leave older values after concurrent writes; delete instead and let reads refill.
"Delete-on-write makes the cache always correct."	A slow reader can re-insert stale data; TTLs bound it.
"Redis LRU is exact."	It is approximated by sampling [S2].
"An expired key frees memory immediately."	Redis expires passively on access and actively by sampling, so there can be a short delay [S4].
"More cache nodes fix a hot key."	One key lives on one node; you need local caching or replication of that key.
"If Redis is down, the app is down."	Only if you designed it that way; treat cache failures as misses and protect the database.
"Write-back is just faster write-through."	It can lose unflushed data; use only for loss-tolerant data.

INTERVIEW MODE: SAY IT LIKE THIS

- Typical caching traps: assuming caches always help, overwriting instead of deleting on writes, believing delete-on-write is race-free, assuming Redis LRU is exact or expiry is instant, scaling out to fix a hot key, letting a cache outage take the app down, and using write-back for data that must not be lost.

TEAR OUT THIS PAGE

One-page cheat sheet

WHAT THEY ASK	THE ANSWER	THE TRAP
Where would you cache?	Name the layer: browser, CDN, proxy, local, Redis, DB	"Add Redis" with no layer or key named
Cache-aside vs read-through	App loads on miss vs cache loads on miss	Forgetting who owns the DB call
Write-through vs write-back	Sync write to both vs write to cache, flush later	Using write-back for payments
On update, what do you do to the cache?	DB first, then DELETE the key; TTL as backstop	Updating the cache value directly
LRU vs LFU	Recency vs frequency (with ageing)	Claiming one is always better
How does Redis evict?	maxmemory + policy; approximate LRU/LFU by sampling	Saying it keeps an exact LRU list
Thundering herd / stampede	Jitter, lock/single-flight, stale-while-revalidate, early refresh	Only "increase TTL"
Hot key	L1 local cache, key replication, CDN, rate limit	"Add more Redis nodes"
Redis goes down?	Misses not errors, short timeouts, protect DB, replicas, warm up	Letting every request fail
Negative caching	Cache "not found" with a short TTL	Long TTL hides newly created items

1.0 ms
average read at 90% hits (0.5 ms cache, 5 ms DB)

77/s
sho.rt DB reads at 99% hits (vs 772 at 90%)

rate × rebuild
duplicate queries in a stampede (20,000 × 0.05 = 1,000)

INTERVIEW MODE: SAY IT LIKE THIS

- Revise the layers, the four strategies, delete-on-write with TTLs, LRU versus LFU, Redis's sampled eviction, stampede fixes, hot-key fixes, Redis failure handling and negative caching.

WHAT YOU WILL ACTUALLY CONFIGURE AT WORK

Where this shows up in real systems

Redis configuration [S2]: memory cap, policy and sampling accuracy in `redis.conf`.

```
maxmemory 24gb
maxmemory-policy allkeys-lru
maxmemory-samples 5        # accuracy of approximate LRU
```

Redis locks [S3]: the documented simple locking pattern, released with DEL and auto-released by the expiry.

```
SET resource-name anystring NX EX max-lock-time
DEL resource-name
```

nginx [S8]: only one request populates a new cache entry; serve stale content while refreshing in the background.

```
proxy_cache_lock on;
proxy_cache_use_stale updating;
proxy_cache_background_update on;
```

HTTP [S7]: fresh for 600 s, then may be served stale for 30 more seconds while revalidating asynchronously.

```
Cache-Control: max-age=600, stale-while-revalidate=30
```

AT VERY LARGE SCALE: FACEBOOK MEMCACHE [S5]

Leases to stop stale sets and thundering herds, and separate **"gutter" servers** that take over for failed nodes rather than rehashing keys (which could overload a node that inherits a hot key).

INTERVIEW MODE: SAY IT LIKE THIS

- In production this appears as Redis maxmemory policies and NX-EX locks, nginx cache locks and stale-while-updating, HTTP stale-while-revalidate headers, and Facebook's memcache leases and gutter pools.

FUNDAMENTALS WITH MODEL ANSWERS

Interview questions 1 and 2

Two questions are Source-confirmed (Amazon, Razorpay); the rest are Pattern-only, so no company name is attached. Labels are kept exactly as the evidence allows.

Q1 Explain how an LRU cache works and how you'd implement get and put in O(1).

SOURCE-CONFIRMED

Amazon, Software Engineer, Hyderabad, technical screen (reported June 2026): the interviewer pressed on combining a hash map with a doubly linked list and on how eviction works [S10].

Model answer. "A hash map gives O(1) lookup from key to a node; a doubly linked list keeps nodes in recency order. On get, I find the node through the map and move it to the front. On put, I insert or update at the front; if size exceeds capacity, I remove the node at the tail, which is least recently used, and delete its key from the map. Both operations are O(1) because unlinking a node in a doubly linked list doesn't need a traversal."

> **Why a doubly linked list, not singly?** Removing a node from the middle needs its predecessor; with a singly linked list that is O(n).

> **Capacity 0, or put on an existing key?** Capacity 0 stores nothing; an existing key updates the value and moves to the front without eviction.

> **Make it thread-safe and distributed.** One lock (or striped locks) for a local cache; at scale use Redis with allkeys-lru, which approximates LRU by sampling rather than keeping a global list [S2].

Weak: "Use a queue and remove from the front."

Strong: "Hash map plus doubly linked list, O(1) both ways, and here's how Redis approximates it."

Q2 You use Redis as a cache. What if Redis goes down? Is stale data a problem?

SOURCE-CONFIRMED

Razorpay, Senior Software Engineer, HLD discussion (April 2025): asked what Redis is used for, about stale data in the cache layer, and what happens if Redis goes down [S9].

Model answer. "The cache is an optimisation, not the source of truth, so cache failures become misses: short timeouts on cache calls, fall back to the database. To stop the database from being flattened, I keep a small local cache for the hottest keys, a circuit breaker and rate limits in front of the database, and run Redis with replicas and automatic failover. For staleness, every key has a TTL, writes delete the key after updating the database, and data that must never be stale isn't served from the cache."

> **Why short timeouts on the cache?** A hung Redis with a long timeout makes every request slow; a fast failure lets you serve from the database.

> **7,716 reads/s normally 99% cached; Redis dies.** Database load jumps from about 77 to 7,716 reads/s, 100×; plan capacity or shed load.

> **Redis comes back empty.** Warm the hottest keys first and let the rest fill gradually; jittered TTLs avoid synchronized expiry later.

Weak: "Redis never goes down."

Strong: "Misses not errors, protect the database, replicas, warm-up, bounded staleness."

FUNDAMENTALS WITH MODEL ANSWERS

Interview questions 3 to 5

Q3 Compare cache-aside, write-through and write-back. PATTERN-ONLY

Model answer. "Cache-aside: the app reads the cache, loads from the database on a miss and deletes the key on writes; simple and resilient, with a stale window. Write-through: every write goes to cache and database synchronously, so written data is fresh but writes are slower and the cache fills with data that may never be read. Write-back: writes land in the cache and are flushed later; fastest writes, but unflushed data can be lost, so only for things like counters."

> **Which for a user-profile service?** Cache-aside with delete-on-write.

> **Write-back node crashes; what's lost?** Everything written since the last flush.

> **How would you make write-back safer?** Replicate the cache, flush often, or log writes durably before acknowledging, which reduces the speed advantage.

Q4 How do you keep a cache consistent with the database? PATTERN-ONLY

Model answer. "Database first, then delete the cache key, and a TTL on everything as a bound. I explain the race where a slow reader re-inserts an old value after the delete, and the defences: delayed double delete, versioned writes or leases, or invalidation from the database change log. And I decide per data type how much staleness is acceptable; for balances, I read the database."

> **Why delete rather than update?** Concurrent updates can leave the older value in the cache; a delete forces a fresh read.

> **What if the delete fails?** The TTL bounds staleness; retry the delete asynchronously; or drive invalidation from the change log.

> **Multiple regions?** Invalidate in every region's cache, typically from a replicated change stream; accept a short cross-region stale window.

Q5 What is a cache stampede and how do you prevent it? PATTERN-ONLY

Model answer. "A popular key expires and many requests miss at once and all rebuild it, overloading the database. I spread expiries with TTL jitter, let one request rebuild behind a short-lived lock or request coalescing while others wait or get slightly stale data, and for very hot keys refresh before expiry, either in the background or probabilistically like XFetch."

> **What if the lock holder crashes?** The lock has an expiry (SET … NX EX), so another request takes over [S3].

> **20,000 req/s, 50 ms rebuild.** About 1,000 duplicate queries without protection; 1 with a lock.

> **At the HTTP layer?** stale-while-revalidate or nginx proxy_cache_lock / use_stale updating [S7, S8].

FUNDAMENTALS WITH MODEL ANSWERS

Interview questions 6 to 8

Q6 One key gets 30% of all traffic and its Redis node is at 100% CPU. What do you do?

PATTERN-ONLY

Model answer. "That's a hot key: adding nodes doesn't help because one key lives on one node. I add a short-TTL local cache in each app server for that key, or replicate it under several suffixed keys spread across nodes, cache it at the CDN if it's public, and rate-limit abuse. I also add per-key metrics so I see the next hot key early."

> **Downside of the local cache?** Staleness up to its TTL and N copies to keep in mind.
>
> **Downside of key replication?** Every write must update all copies.
>
> **How do you detect hot keys?** Per-key counters or sampling, and one node's CPU far above its peers.

Q7 LRU or LFU? How does Redis evict?

PATTERN-ONLY

Model answer. "LRU evicts what hasn't been used recently, good for recency-driven traffic; LFU evicts what's used least often, good for long-lived popularity, but it needs ageing so old favourites don't stay forever. Redis evicts when it hits maxmemory, using a policy like allkeys-lru or allkeys-lfu; both are approximations based on sampling a few keys, and LFU counters decay over time."

> **Difference between eviction and expiry in Redis?** Eviction is triggered by memory pressure under maxmemory; expiry by TTL, passively on access and actively in the background [S2, S4].

Q8 Local cache or Redis?

PATTERN-ONLY

Model answer. "Local is fastest and needs no network, but each server has its own copy, invalidation must reach every server and it's cold after each deploy. Redis is shared, invalidated with one delete, and survives app restarts, but adds a hop and a dependency. For hot, rarely changing data I use a short-TTL local cache in front of Redis."

> **What would make you skip the local cache entirely?** Data that changes often or must be consistent across servers immediately; one shared copy in Redis is simpler to keep correct.

INTERVIEW MODE: SAY IT LIKE THIS

- Expect LRU implementation and policy, Redis failure and staleness, the four strategies, consistency and invalidation races, stampede prevention, hot keys, eviction policies and the local-versus-distributed choice.

HOW TO THINK WHEN IT BREAKS

Scenario questions 1 to 5

Original troubleshooting scenarios (no company source for these exact situations).

S1. After a deploy, database CPU spikes to 100% for ten minutes and then recovers.

1. Suspect cold local caches: every app server restarted with an empty L1 cache.
2. Confirm: miss rate on L1 near 100% right after deploy, database QPS spike aligned with rollout.
3. Fix: roll out gradually, warm caches before taking traffic, rely more on Redis (which survives deploys) for hot data.

Deploying nine servers with empty caches at once is the backend version of everyone opening the Tatkal booking page at 10:00:00.

S2. Users report that an edited link still redirects to the old URL for hours.

1. Check the write path: is the cache key deleted after the database update? Is it the right key name?
2. Check other writers (admin tools, batch jobs) that bypass the delete.
3. Check the TTL: if keys were re-cached with a plain SET without EX, they now have no expiry [S3].
4. Fix the missing delete; add a TTL to every key; consider change-log-driven invalidation.

S3. Redis memory is full and writes are failing.

1. maxmemory-policy is probably noeviction, so new writes error out [S2].
2. For a pure cache, switch to allkeys-lru; find keys without TTLs; check for unbounded key growth (for example per-request keys).

S4. Every day at exactly 00:00, latency spikes for two minutes.

1. Something caches many keys at the same time with the same TTL (a nightly job), so they all expire together.
2. Add TTL jitter; refresh hot keys in the background before expiry.

S5. One Redis node is at 95% CPU while the others are at 20%.

1. Hot key or a big key on that node: check per-key traffic and key sizes.
2. Hot key: local cache or key replication. Big key (a huge list or hash): split it.

HOW TO THINK WHEN IT BREAKS

Scenario questions 6 and 7 and the pattern

S6. Bots requesting random short codes are hammering the database.

1. Every unknown code is a miss that goes to the database.
2. Add negative caching with a short TTL; rate-limit by client; optionally a Bloom filter of existing codes to reject impossible ones before the cache.

S7. Cache hit rate dropped from 95% to 60% after a feature launch.

1. Did the key format change (for example a new parameter in the key)? Each variant is a separate entry.
2. Did the hot set grow beyond memory, causing evictions? Check eviction counters.
3. Normalise keys, increase memory, or cache at a coarser level.

THE PATTERN BEHIND EVERY SCENARIO

When something breaks, **ask what changed** (a deploy, a key format, a nightly job), **look at the hit rate and the evictions**, then check the **write path** and the **TTLs**. Caching bugs are almost always one of those four.

SYMPTOM	FIRST SUSPECT	CONCEPT
DB spike right after a deploy	Cold local (L1) caches	8
Edited data stays old for hours	Missing DEL, or a key with no TTL	2, 5
Redis full, writes failing	noeviction, keys without TTLs	4
Spike at the same time every day	Synchronized expiry	6
One Redis node hot	Hot key or big key	7
Random-code floods	No negative caching	Worked example
Hit rate drops after a launch	Key format change, hot set outgrew memory	1, 4

INTERVIEW MODE: SAY IT LIKE THIS

- For caching incidents I check cold caches after deploys, missing invalidations and keys without TTLs, the eviction policy, synchronized expiries, hot or oversized keys, negative-lookup floods and key-format changes that destroy the hit rate.

SOLVE THESE BEFORE YOU LOOK

Practice problems 1 to 4

Original problems, not reported questions. Cover the solution, try it on paper, then check.

P1 Hit rate for a target EASY, NUMERICAL

Cache hit costs 1 ms, database read costs 20 ms, a miss pays both. What hit rate gives a 3 ms average?

> **Hint.** average = h × 1 + (1 − h) × 21.

SOLUTION

h + 21 − 21h = 3 → 21 − 20h = 3 → h = 0.9. **A 90% hit rate.**

P2 Trace LRU EASY

Trace LRU with capacity 2 on the sequence X Y X Z Y X. Count hits.

> **Hint.** Keep the order of last use.

SOLUTION

X(miss) [X]; Y(miss) [X,Y]; X(hit) [Y,X]; Z(miss, evict Y) [X,Z]; Y(miss, evict X) [Z,Y]; X(miss, evict Z) [Y,X]. **1 hit.**

P3 Size the stampede MEDIUM, NUMERICAL

A train-ticket enquiry API gets 12,000 requests/s for the same popular route at 10 am; rebuilding the cached availability summary takes 80 ms. How many duplicate database queries happen at expiry without protection, and how many with a lock?

> **Hint.** requests per second × rebuild time.

SOLUTION

12,000 × 0.08 = **960** duplicate queries without protection; **1** with a lock (plus a little waiting for the others).

P4 Invalidate results safely MEDIUM

A college results portal caches each student's result for 1 hour. Results are corrected occasionally. Design the invalidation, including what happens if the correction job runs while students are reading.

> **Hint.** Database first, then delete; think about the slow-reader race and a bound.

SOLUTION

The correction job updates the database row, then deletes `result:{roll}`. A reader that read the old row just before the update could re-insert it after the delete; mitigate with a delayed second delete (for example after 2 seconds) and keep the 1-hour TTL as a bound. If corrections must appear instantly, reduce the TTL for recently corrected rows or read results straight from the database for a short window after correction.

SOLVE THESE BEFORE YOU LOOK

Practice problems 5 and 6

P5 Layers for skewed traffic MEDIUM-HARD

A food-ordering app shows a restaurant's menu. 5,000 restaurants, menus change a few times a day, the top 50 restaurants get 60% of traffic. Propose layers, keys, TTLs and hot-key handling.

> **Hint.** Different restaurants have very different traffic; consider L1 for the top ones.

SOLUTION

Redis cache-aside with key `menu:{restaurant_id}:{version}` or delete-on-update; TTL 30 minutes ± 5 minutes jitter as a bound; for the top 50, an L1 local cache with a 10-second TTL on every app server so their 60% of traffic rarely touches Redis; publish menu updates as events that delete the Redis key (and optionally broadcast an L1 purge). Images go through the CDN. Staleness is bounded by 10 seconds for L1 and by delete-on-update for Redis.

P6 The policy that cannot evict HARD

Your Redis node holds 40 GB of cached data and maxmemory-policy is volatile-lru, but half your keys were written without TTLs. Memory hits the limit. What happens, and how do you fix it without downtime?

> **Hint.** What can volatile-lru evict [S2]?

SOLUTION

volatile-lru only evicts keys that have a TTL. Keys without TTLs can never be evicted, so once only they remain, Redis cannot free memory and new writes fail like noeviction. Fix: switch to allkeys-lru with `CONFIG SET maxmemory-policy allkeys-lru` (no restart needed), fix the code to always set TTLs, and gradually add TTLs to existing keys with a background SCAN plus EXPIRE job.

INTERVIEW MODE: SAY IT LIKE THIS

- Practice covers hit-rate arithmetic, eviction traces, stampede sizing, invalidation under concurrency, layered design for skewed traffic, and Redis eviction-policy failure modes.

TEN QUESTIONS, TWO MINUTES EACH

Mini quiz

Original questions. Cover the answers, answer all ten, then check.

1. Cache-aside on write:
(a) update cache then DB (b) update DB then delete cache key (c) update cache only (d) delete cache then never update DB
Answer: b. (a) risks the cache holding data the DB never stored; (c) is write-back; (d) loses the write.

2. Redis allkeys-lru evicts:
(a) exactly the least recently used key (b) an approximation by sampling (c) random keys (d) keys with TTLs only
Answer: b [S2]. (d) describes volatile policies.

3. A stampede is best prevented by:
(a) longer TTL only (b) rebuild lock plus jitter (c) more app servers (d) disabling the cache
Answer: b. (a) delays it; (c) adds more requesters; (d) moves all load to the DB.

4. A hot key is fixed by:
(a) adding Redis nodes (b) a short-TTL local cache (c) a longer TTL (d) noeviction
Answer: b. One key still lives on one node, so (a) doesn't help.

5. Write-back is acceptable for:
(a) bank balances (b) click counters (c) order payments (d) passwords
Answer: b. Losing a few seconds of counts is tolerable.

6. Plain SET on a key that had a TTL:
(a) keeps the TTL (b) discards the TTL (c) doubles it (d) errors
Answer: b [S3].

7. 90% hit rate, 0.5 ms cache, 5 ms DB, miss pays both: average is
(a) 0.5 ms (b) 1.0 ms (c) 5 ms (d) 5.5 ms
Answer: b. 0.45 + 0.55 = 1.0 ms.

8. Cache-Control: max-age=600, stale-while-revalidate=30 allows serving stale for up to:
(a) 600 s (b) 30 s after expiry while revalidating (c) forever (d) 0 s
Answer: b [S7].

9. Negative caching should use:
(a) a long TTL (b) a short TTL (c) no TTL (d) write-back
Answer: b. A long TTL would hide newly created items.

10. In Redis, an expired key is removed:
(a) only when memory is full (b) passively on access and actively by background sampling (c) at midnight (d) never
Answer: b [S4].

INTERVIEW MODE: SAY IT LIKE THIS

- Quick facts: database first then delete; Redis LRU is sampled; locks and jitter stop stampedes; local caches fix hot keys; write-back only for loss-tolerant data; SET clears TTLs; stale-while-revalidate allows a grace period; negative caches use short TTLs; Redis expires keys passively and actively.

FIVE SENTENCES TO KEEP

Recap and the 60-second answer

1. Caches live at many layers; say which one and why, and watch the hit rate.
2. Cache-aside is the default; write-through keeps written data fresh; write-back is fast but can lose data.
3. Eviction (LRU, LFU, FIFO) handles a full cache; TTLs handle time; Redis approximates both.
4. Invalidate by updating the database then deleting the key, with a TTL as a backstop for the race that survives.
5. Protect the database from stampedes (jitter, locks, stale-while-revalidate, early refresh), handle hot keys with local caches or replication, and treat a cache outage as misses, not errors.

SAY THIS IN AN INTERVIEW: "HOW WOULD YOU ADD CACHING TO THIS SYSTEM?"

"First I'd identify the hot reads and how stale each can be. I'd use cache-aside with Redis: read the cache, fall back to the database on a miss and fill the cache with a TTL plus some jitter; on writes, update the database and then delete the key. I'd size the cache to the hot set and use an LRU-style eviction policy. To protect the database I'd add a rebuild lock or request coalescing for popular keys, maybe stale-while-revalidate, and negative caching for missing items. For extreme hot keys I'd add a short-TTL local cache on each app server. If Redis fails, cache errors become misses with short timeouts, and the database is protected by rate limits while a replica takes over."

CHECKLIST ITEM	COVERED WHERE
Where caches live	Concept 1, source diagram 1
Cache-aside	Concept 2, source diagram 2, worked example
Read-through, write-through, write-back	Concept 3, Q3
Eviction: LRU, LFU, TTL	Concept 4, Q1, Q7, P2, P6
Cache invalidation	Concept 5, Q4, S2, P4
Thundering herd and cache stampede	Concept 6, source diagram 3, Q5, S4, P3
Hot keys	Concept 7, Q6, S5
Redis vs local cache	Concept 8, Q2, Q8, S1

INTERVIEW MODE: SAY IT LIKE THIS

- Caching design is choosing the layer, the strategy, the TTL and eviction, the invalidation path, stampede and hot-key protection, and the failure plan.

CHECK OUR WORK

Sources and honest gaps

S = a page opened in full. Redis and nginx behaviour comes from their official documentation; stampede and lease techniques from the original papers and RFC; company questions from dated candidate reports. Every calculation in this lesson was recomputed in code when this PDF was built.

ID	PAGE	WHAT IT SUPPORTED	URL
S1	Engineering at Meta: "More Details on Today's Outage" (23 Sep 2010)	2.5-hour outage, invalidation feedback loop, recovery by stopping traffic	engineering.fb.com/2010/09/23/uncategorized/more-details-on-today-s-outage
S2	Redis docs: Key eviction	maxmemory, all policies, allkeys-lru as a good default, sampled LRU, LFU with decay	redis.io/docs/latest/develop/reference/eviction/
S3	Redis docs: SET	EX/PX/NX options, SET discards previous TTL, SET NX EX locking pattern	redis.io/docs/latest/commands/set/
S4	Redis docs: EXPIRE	TTL semantics, overwrite clears TTL, passive and active expiry	redis.io/docs/latest/commands/expire/
S5	Nishtala et al., "Scaling Memcache at Facebook", NSDI 2013	Leases (token at most once per 10 s per key), peak DB rate with leases, single key 20% of a server's requests, gutter pools	usenix.org/system/files/conference/nsdi13/nsdi13-final170_update.pdf
S6	Vattani, Chierichetti, Lowenstein, "Optimal Probabilistic Cache Stampede Prevention", VLDB 2015	Probabilistic early expiration and XFetch	vldb.org/pvldb/vol8/p886-vattani.pdf
S7	RFC 5861: HTTP Cache-Control Extensions for Stale Content	stale-while-revalidate and stale-if-error semantics and example	rfc-editor.org/rfc/rfc5861
S8	nginx docs: ngx_http_proxy_module	proxy_cache_lock (default off, 5 s timeouts), use_stale updating, background_update	nginx.org/en/docs/http/ngx_http_proxy_module.html
S9	LeetCode Discuss: Razorpay Senior Software Engineer, April 2025	Redis usage, stale data in the cache layer, Redis going down	leetcode.com/discuss/interview-experience/6651036
S10	PracHub: Amazon Software Engineer, LRU cache (reported Jun 2026, published 5 Sep 2026)	LRU with hash map and doubly linked list, eviction details	prachub.com/interview-experiences/amazon-software-engineer-interview-experience-lru-cache-implementation-details-4f15bf6e7e
S11	Hello Interview: Caching (core concept)	Hot keys and Redis-failure framing; cross-check only, not for company claims	hellointerview.com/learn/system-design/core-concepts/caching

Touched lightly or left out (interviewers sometimes reach these)

- LRU and LFU implementation code and thread safety: LLD Lesson 19.
- Redis persistence (RDB snapshots, AOF), Redis Cluster internals and replication details.
- Cache consistency across regions and CDN purge mechanics in depth.
- Distributed locks with strong guarantees (the simple SET NX EX lock can misbehave under clock pauses or failover; that debate is beyond this lesson).
- Company evidence: dated reports confirm LRU questions at Amazon and Redis-failure and staleness questions at Razorpay; no dated caching question was found for Uber, Swiggy, Flipkart or PhonePe software-engineer loops in the window.

Interview mode. This lesson covers caching design end to end; LRU code comes in LLD, and distributed-lock correctness and Redis persistence are deeper topics.

SELF-CHECK

Can you say yes to all four?

I can name the layer
Browser, CDN, proxy, local, Redis, DB, and the hit-rate sum.

I can pick a strategy
Cache-aside by default; write-back only for counters.

I can invalidate safely
DB first, then DEL, TTL on everything, and the race.

I can protect the database
Stampedes, hot keys, and Redis going down.

> "How would you add caching to this system?" **Now you answer with the layer, the strategy, the TTL, the invalidation path, and what protects the database.**

YOUR NEXT STEPS

STEP 01
Solve problems 1 to 6 on paper

STEP 02
Trace LRU on 3 sequences of your own

STEP 03
Read ahead: Lesson 7, scaling databases